PROBLEM & SCOPE

One clear question, answered completely

A precise problem statement, and the boundary of what this build actually covers.

PROBLEM STATEMENT

Given a token stream for a restricted, C-like language, decide whether it is syntactically valid under a fixed grammar.

When it is not valid, report **every** syntax error found in a single pass — the line, what was expected, and what was actually found — instead of stopping at the first mistake, while still producing a usable parse tree for everything that *did* match.

No lexer is implied by this statement — tokenizing raw source text is a separate, later problem.

IN SCOPE

- Declarations, assignment, `if`/`else`, `while`, `print`, arithmetic
- A symbolic **token stream** as input — lexing is assumed already done
- Full LL(1) analysis: FIRST, FOLLOW, and a conflict-checked parsing table
- Two syntax-error recovery strategies, working together
- Parse-tree construction plus a step-by-step visual trace of the run

Semantic checks — types, scope, values — are intentionally out of scope; see Limits & Extensions.

GRAMMAR SPECIFICATION

Thirteen rules define the entire language

Written once as pure BNF, so the same rules drive both the grammar and the algorithms that reason about it.

```
program     -> statement*
statement   -> declStmt | assignStmt | ifStmt
             | whileStmt | block | printStmt
declStmt    -> "int" ID ";"
assignStmt  -> ID "=" expr ";"
ifStmt      -> "if" "(" cond ")" block
               ( "else" block )?
whileStmt   -> "while" "(" cond ")" block
block       -> "{" statement* "}"
printStmt   -> "print" "(" expr ")" ";"
cond        -> expr relop expr
expr        -> term (("+"|"-") term)*
term        -> factor (("*"|"/") factor)*
factor      -> ID | NUM | "(" expr ")"
```

SYMBOL INVENTORY

17	21	0
NON-TERMINALS	TERMINALS	LEFT RECURSION

The * and ? operators shown here are surface sugar. Before FIRST/FOLLOW/LL(1) construction runs, they are expanded by hand into pure right-recursive rules with explicit epsilon productions (e.g. `exprTail -> "+" term exprTail | ε`) — the textbook algorithms are defined over pure BNF, not EBNF.

start symbol: program · no ambiguity, no backtracking

ALGORITHM DESIGN

Correctness comes from the table, not a hunch

No DFA states and no precedence relations here — the decision procedure is a predictive LL(1) table, built mechanically, then mirrored by hand in recursive descent.

1

FIRST(X)

Fixed-point closure: every terminal that could be the first token of anything X produces.

2

FOLLOW(X)

Fixed-point closure: every terminal legally allowed to appear immediately after X.

3

Table build

Each production is registered at (X, terminal) for every terminal in its FIRST — and in FOLLOW(X) if it can vanish to ε.

4

Conflict check

A second production ever claiming an occupied cell raises immediately — a FIRST/FIRST or FIRST/FOLLOW conflict.

75 / 75 table cells fill with zero conflicts → the grammar is provably LL(1): one token of lookahead always selects exactly one production, deterministically, with no backtracking.

The recursive-descent parser does not consult this table at runtime — each of its functions (`parse_if_stmt`, `parse_expr`, …) encodes the same FIRST-set dispatch logic directly in code. The table's job is to **prove**, once, that those hand-written decisions can never collide.